The Building Blocks of LLMs

Five Questions, One Sentence, 2,000 Years of Mathematics

From “The cat sat on the ____” to the Transformer

Prof. J. Osterrieder

2026

One Sentence. Five Questions.

"The cat sat on the _____"

Every time you use ChatGPT or Claude, the AI is doing **one thing**: predicting the next word.

To play this game, it needs five abilities:

1. How does it know which words are similar?
2. How does it choose the next word?
3. How does it learn from mistakes?
4. How does it understand context?
5. How does it get so good?

Each was invented by specific people, at specific times, for reasons that had **nothing to do with AI**.

“The cat sat on the _____” – How Does It Know Which Words Are Similar?

“The cat sat on the _____”

The AI thinks “mat” and “rug” are good answers, but not “quantum.” How does it know which words are similar?

Good candidates:

✔ mat ✔ rug ✔ carpet

Bad candidate:

✘ quantum

The secret: every word is a list of numbers.

“mat” $= [0.2,\ 0.8,\ -0.1,\ \ldots]$
(768 numbers)

Similar words $\rightarrow$ similar lists.

The Inventors of the Number-World

Grassmann, 1844

A schoolteacher in Prussia.

Published a book so abstract that **nobody read it**.

Invented **vector spaces** – the exact structure AI uses.

Cayley, 1858

Sylvester named it "matrix" – Latin for *womb*.

Cayley defined **matrix multiplication**.

Every neural network layer uses it.

Gibbs, 1881

Yale physicist.

Formalized the **dot product**.

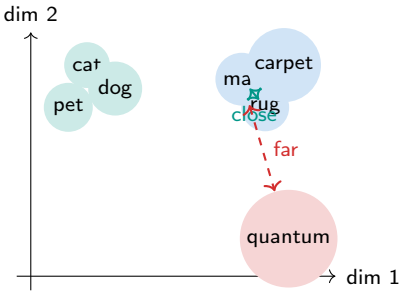
Multiply, add, get similarity.

$$\vec{a} \cdot \vec{b} = \sum a_i b_i$$

Key Insight

None of them had any idea their math would end up inside your phone.

Words Live in a Number-World



Dot product = similarity

$\vec{\text{mat}} \cdot \vec{\text{rug}} \quad = 0.92$ (similar)

$\vec{\text{mat}} \cdot \vec{\text{quantum}} \quad = 0.03$ (unrelated)

👉 Interactive Moment:

king − man + woman = ?

Mikolov (Word2Vec, 2013): word vector arithmetic works.

"Not a trick. Geometry in 768 dimensions."

Every Layer Is a Matrix Multiplication

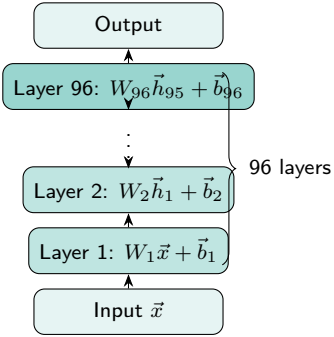
One layer:

$$\vec{y} = W \cdot \vec{x} + \vec{b}$$

Input vector $\vec{x}$, weight matrix W, bias $\vec{b}$, output $\vec{y}$.

A neural network:

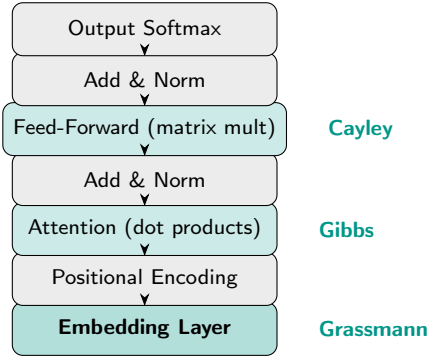
Stack 96 of these.



Key Insight

Cayley's math from 1858, repeated 96 times, is the engine of modern AI.

Building Block Found: Vectors, Matrices, Dot Product



Where Topic 1 lives:

- **Embedding layer** – words become vectors (Grassmann)
- **Attention** – dot products measure similarity (Gibbs)
- **Every layer** – matrix multiplication (Cayley)

Topics explored: 1 / 5

*We know which words are similar. Now: how does the AI **choose**?*

"The cat sat on the _____" – How Does It Choose the Next Word?

"The cat sat on the _____"

The AI has 50,000+ words to choose from. It picks "mat" 68% of the time. How?

Raw scores (logits):

mat	3.2
floor	1.8
table	1.1
couch	0.6
quantum	-4.7

Probabilities (???):

mat	68%
floor	15%
table	8%
couch	4%
quantum	$\approx 0\%$

How do raw numbers become a probability distribution?

From a Gambling Question to Every Word AI Says

Pascal & Fermat, 1654

A gambler lost at dice and wrote to Pascal.

Probability was born from a **letter exchange**.

Boltzmann, 1870s

Studied how gas molecules distribute among energy levels.

His formula: $\frac{e^{x_i}}{\sum e^{x_j}}$

Now called softmax.

Shannon, 1948

Language as statistics.

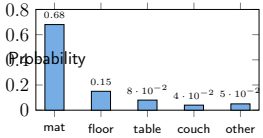
Cross-entropy: how to measure prediction quality.

Key Insight

Pascal's gambling question → Boltzmann's gas molecules → the AI choosing "mat."

The Softmax Pipeline

Raw scores $\xrightarrow{\text{exponentiate}}$ e^{z_i} $\xrightarrow{\text{normalize}}$ $\div \sum e^{z_j}$ $\xrightarrow{\text{sum to 1}}$ **Probabilities**



$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

👉 **Interactive Moment:**

"The cat sat on the _____"

What word comes next?

You and the AI are doing the same thing.

Measuring Mistakes: Cross-Entropy

The correct answer was "mat."

Scenario A: AI said "mat" with 80%
Loss $= -\log(0.80) = 0.22$ ✔ Good

Scenario B: AI said "mat" with 5%
Loss $= -\log(0.05) = 3.00$ ✖ Terrible

$$\mathcal{L} = -\log p(\text{correct word})$$

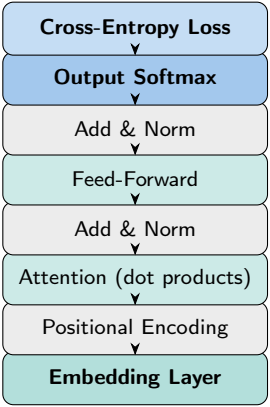
Shannon, 1948:

- More surprised by the truth → higher loss
- Less surprised → lower loss
- This is the **only** number the AI tries to minimize

Key Insight

Cross-entropy connects **prediction** (Topic 2) to **learning** (Topic 3).

Building Block Found: Probability, Softmax, Cross-Entropy



Shannon

Boltzmann

Where Topic 2 lives:

- **Output softmax** – scores become probabilities (Boltzmann)
- **Loss function** – cross-entropy measures mistakes (Shannon)

Topics explored: 2 / 5

The AI made a wrong prediction. Cross-entropy measured the mistake. But how does the correction travel ***backward*** *through 96 layers?*

“The cat sat on the ____” – The AI Said “Dog.” How Does It Learn?

“The cat sat on the _____”

The AI said “dog.” That’s wrong. How does the correction travel backward through 96 layers and 175 billion parameters?

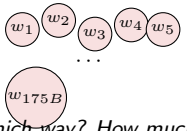
The prediction:

“The cat sat on the dog” ✗

Cross-entropy loss: **high**

The AI needs to adjust **175 billion parameters** so “mat” scores higher next time.

175 billion tiny knobs

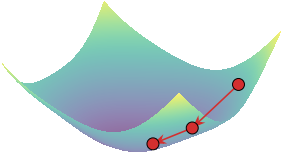


Which way? How much?

Gradient Descent: Compute the Slope, Step Downhill, Repeat

Cauchy, 1847 (Paris):

If you can compute the **slope** at any point, you can find the bottom by always stepping **downhill**.



The Recipe:

1. Where am I? (current loss)
2. Which way is downhill? (derivative)
3. Take a step. (learning rate $\times$ gradient)
4. Repeat. (billions of times)

Newton & Leibniz (1680s) gave us the derivative. Cauchy gave us the **algorithm**.

The Chain Rule Trains Every AI

The chain rule:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$$

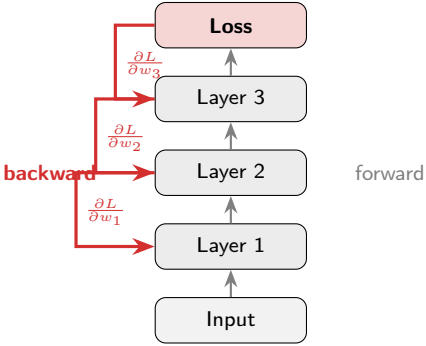
Derivative of a composition = product of individual derivatives.

In a network:

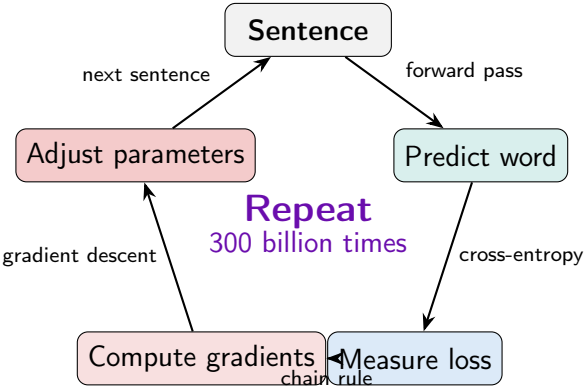
Error flows **backward**, layer by layer. Each layer learns its contribution to the mistake.

1970 Linnainmaa: auto-differentiation

1986 Hinton: backprop



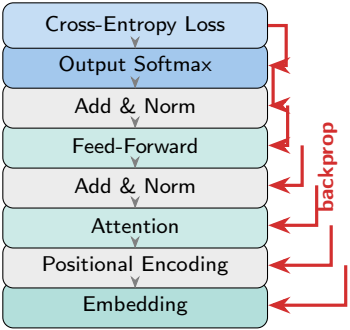
The Training Loop



Key Insight

GPT-3 ran this loop on 300 billion tokens. Same math every time: Topics 1 + 2 + 3.

Building Block Found: Derivatives, Gradient Descent, Backpropagation



Where Topic 3 lives:

- **Backward pass** – gradient descent (Cauchy, 1847)
- **Chain rule** – error flows through layers (Linnainmaa / Hinton)

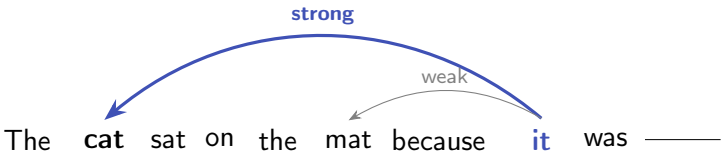
Topics explored: 3 / 5

*The AI can represent, predict, and learn. But it has no notion of **context**.*

"The cat sat on the mat because IT was ____" – What Does "It" Mean?

"The cat sat on the _____"

The word "it" could mean "cat" or "mat." The AI needs to look at the WHOLE sentence to decide. How?



Before 2014, AI processed words one at a time. Early words were **forgotten** after ∼50 words.

From Poetry Analysis to "Attention Is All You Need"

Markov, 1906

Analyzed Pushkin's poetry.

Markov chains: next state depends on current state.

But chains **forget**.

Bahdanau, 2014

Montreal. Translation.

"Why compress into one vector?

Look back at the whole input."

Invented **attention**.

Vaswani et al., 2017

Google. 8 authors.

"**Attention Is All You Need**"

Architecture using **only** attention.

The **transformer**.

Key Insight

Markov started it. Bahdanau cracked the memory problem. Vaswani assembled the machine.

Every Word Looks at Every Other Word

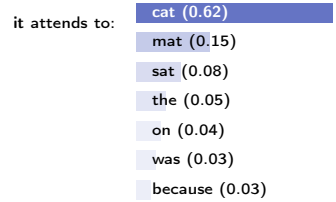
Self-Attention: Q, K, V

- **Q**uery: "What am I looking for?"
- **K**ey: "What do I contain?"
- **V**alue: "What information do I carry?"

Attention score = dot product of Q and K (Gibbs, T1!)
Then softmax turns scores into weights (Boltzmann, T2!)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

Attention heatmap:



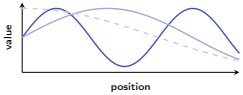
"it" looks back at "cat" and assigns high weight.

Context resolved.

Fourier's Sine Waves Tell the AI Where Each Word Is

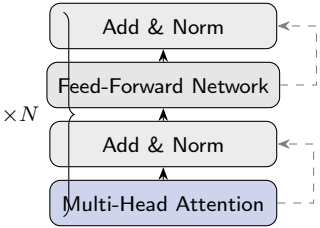
The problem: Self-attention has no sense of word **order**.

The solution: Positional encoding using sine and cosine waves at different frequencies.



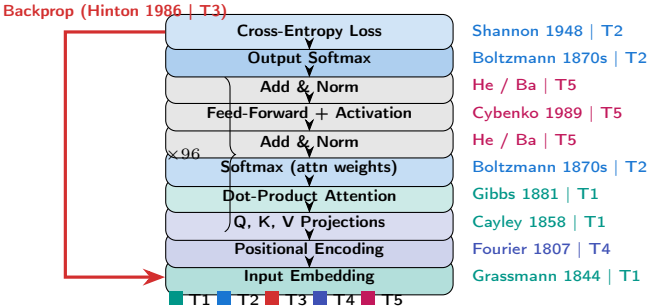
Fourier, 1807: decomposed heat into waves.
Vaswani, 2017: decomposed position into waves.
Same math, 210 years apart.

One Transformer Layer:



Stack $N = 96$ of these $\rightarrow$ GPT-4.

2,000 Years of Mathematics in One Diagram



Building Block Found: Attention, Self-Attention, Positional Encoding

The transformer IS Topics 1–4 combined:

- T1: Embeddings, dot products, matrix mult
- T2: Softmax, cross-entropy
- T3: Backpropagation, gradient descent
- T4: Attention, position, the architecture

Vaswani et al. (2017): 8 authors assembled 2,000 years of math into one architecture.

Topics explored: 4 / 5

The five questions:

- [x] Similar words
- [x] Choose next word
- [x] Learn from mistakes
- [x] Understand context
- [ ] Get so good

The machine works. But why do 96 layers work so much better than one?

“The cat sat on the _____” – One Layer Is Dumb. 96 Layers Are Brilliant. Why?

“The cat sat on the _____”

One layer is mediocre. 96 layers are brilliant. 175B parameters. How does scale turn simple math into something that feels like intelligence?

1 layer predicts:

the	22%
a	18%
was	12%

Vague. Generic. Useless.

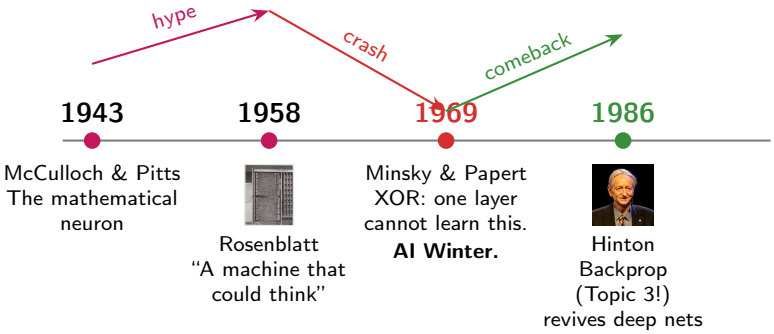
96 layers predict:

mat	68%
floor	15%
table	8%

Precise. Contextual. Useful.

Same architecture. Same math. Just **deeper** and **bigger**. Why does that work?

1943: Invention. 1969: Crash. 1986: Comeback.



Key Insight

The solution: **add more layers**. But nobody knew how to train them until Hinton showed the chain rule (Topic 3) could do it.

Three Tricks That Made Deep Networks Possible

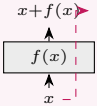
Activation

Without it, 96 layers = 1 layer.
Nonlinearity matters.

ReLU: $\overline{\max}(0, x)$

Residual (He, 2015)

Signal fades through layers.
Skip connections fix it.
out $= x + f(x)$



LayerNorm (Ba, 2016)

Numbers drift and explode.
Re-center & re-scale each layer.
$\hat{x} = \frac{x-\mu}{\sigma}$
Stability for 96+ layers.

Inside the LLM

The "Add & Norm" blocks in the transformer. He + Ba = the unsung heroes of depth.

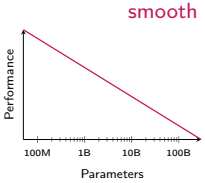
Same Math. Just Bigger.

The GPT Timeline:

Model	Params	Year
GPT-1	117M	2018
GPT-2	1.5B	2019
GPT-3	175B	2020
GPT-4	undisclosed	2023

Same transformer. Same math.
Just **1,500× larger**.

Scaling Laws (Kaplan, 2020):

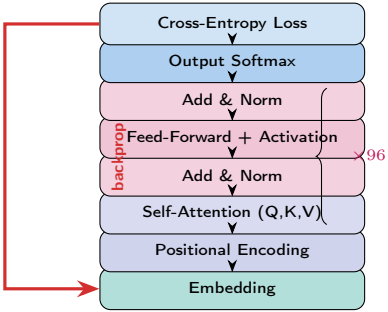


Key Insight

Nobody fully understands why scale works this well. That is an open question.

ALL Building Blocks Found

The transformer – fully colored:



Topics explored: 5 / 5

All five questions answered:

- ☑ Similar words *(vectors)*
- ☑ Choose next word *(softmax)*
- ☑ Learn from mistakes *(backprop)*
- ☑ Understand context *(attention)*
- ☑ Get so good *(depth + scale)*

Every component has a name, a date, an inventor, and a topic where you learned why it matters.

The Story Is Not Over

? Open Question 1

Why do transformers **generalize** so well?

They learn to write poetry from predicting the next word. Nobody fully understands why.

? Open Question 2

What are the **limits** of scaling?

Does making it bigger always make it smarter? Where does it stop?

? Open Question 3

What comes **after** transformers?

The transformer is 8 years old. Every major breakthrough we covered replaced what came before.

Every breakthrough we covered started with someone asking a question nobody else was asking.

The people who answer these questions **might be sitting in this room**.

2,000+ Years. One Machine. Your Turn.

"The cat sat on the _____"

You now know every piece of the machine that answers this question.

Grassmann was a schoolteacher. Pascal answered a gambling question.
Fourier studied heat. Boltzmann studied gas. Markov analyzed poetry.

They followed their curiosity and built the future.

Your turn.